

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-  
a3f908983e45f740d.jsonl`  
- SHA-256: `be0739607cecaacd28b277a49392a0e2712087fcdc0de8d13c9000c5421d0066`  
- Source modified: `2026-07-06T09:14:39+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:13:45.211Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel
creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff
 and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /
 pre-existing. PLAUSIBLE if unsettled. Corrected severity.

FINDING (adr_gate): Ingestion gate auto-watches brand-new channels (opt-OUT), inverting ADR 0009's locked opt-in default
 severity blocker | src/tg_video_filter/telethon_client.py:190

summary: The watched gate only returns early when a source row already exists AND is unwatched (`src is not None and not src.watched`). A brand-new channel has no source row, so `src is None`, the gate falls through, and the message is deduped, filtered, and delivered. The channel is then lazily recorded via `upsert\_channel`, which hardcodes `watched=True` (db.py:1402). Every subsequently seen channel is thus auto-scanned and auto-watched ? the exact opt-out behavior ADR 0009 locks against.

scenario: A freshly-onboarded user with zero watched sources receives a video from any channel/group the account is a member of. Trace: on_new_message -> channel_id computed -> `get\_source\_by\_provider\_id` returns None -> gate condition `None is not None and ...` is False so no early return -> dedup passes -> `engine.evaluate` runs -> `insert\_video` -> the match is DELIVERED to the user's delivery_target. Then `upsert\_channel` inserts the source with watched=True, so all future messages from that channel keep flowing. The user opted into nothing yet receives matches from everything ? the opposite of ADR 0009 line 37-39 ('a freshly-onboarded user scans nothing until they explicitly /watch') and line 130-131 ('New channels discovered after the migration default to watched = 0 and must be explicitly watched'). The first message is also delivered rather than skipped, violating line 44 ('return early ... before any dedup or filter work').

evidence: Empirically reproduced against an in-memory schema (db._apply_schema): BEFORE first message `get\_source\_by\_provider\_id(...)` is None; the gate expression `src is not None and not src.watched` evaluates False (channel is scanned+delivered); AFTER the lazy `upsert\_channel` the source row has watched=True (auto-watched); the 2nd message's gate still evaluates False (keeps scanning). db.py:1402 shows `watched=True` hardcoded in upsert_channel; the gate comment at telethon_client.py:186-188 explicitly acknowledges 'if absent, we let the legacy path upsert it with watched=True for backwards-compatibility' ? but that back-compat carve-out was defined by the ADR only for channels already seen PRE-migration (seen_first_at IS NOT NULL, ADR line 126), not for channels first seen AFTER the migration. Applying watched=True to newly discovered channels contradicts the ADR's new-channel contract.

rec: Make the gate skip unwatched sources by default when no row exists: treat `src is None` as unwatched and return early (create the discoverability row with watched=0 rather than processing). Concretely, change the gate to `if src is None or not src.watched: <lazily upsert the source with watched=0 for /channels discoverability, then> return`. The lazy `upsert\_channel` on the ingestion path must not create rows with watched=True for channels first seen after migration ? only the migration backfill (UPDATE ... WHERE seen_first_at IS NOT NULL) may set watched=1. This restores ADR 0009's 'scan nothing until opt-in' guarantee and moves the early return before dedup/filter/delivery.

2. assistant (2026-07-06T09:13:47.648Z)

I'll verify this finding by reading the actual code and the ADR spec.

3. user (2026-07-06T09:13:49.111Z)

```

1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - **Status**: Accepted
4      - **Date**: 2026-07-04
5      - **Decision owner**: project owner
6
7      ## Context
8
9      Today the userbot scans every channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a record of seen channels;
13     it is not used as a filter.
14
15     The product vision is a multiplexing video re-distribution system:
```

```

16
17 - A small set of high-quality source channels (e.g. spiritual, classical
18 music) are scanned for incoming videos.
19 - Matches are routed to themed destination broadcast channels (e.g. a
20 Reggae channel, a Rap channel) that subscribers opt into.
21 - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23 This requires two capabilities the current architecture does not have:
24
25 1. Selective scanning ? choose which source channels are watched, rather
26 than ingesting all of them.
27 2. Per-source ? per-destination routing ? a match from source channel X is
28 delivered to a specific destination channel Y, instead of the single global
29 `delivery_target` (ADR 0005).
30
31 ## Decision
32
33 Add three additive, independently-shippable capabilities:
34
35 ### 1. Selective scanning (watched-flag)
36
37 Add a `watched` flag to the `channels` table. The default is locked as
38 opt-in: `DEFAULT 0`, so a freshly-onboarded user scans nothing until they
39 explicitly `/watch` a channel. This matches the multiplexing product shape
40 (sources are a curated set) and avoids blindly re-ingesting every chat the
41 account is a member of.
42
43 Gate `on_new_message` at the top: after computing `channel_id`, look up the
44 row and `return` early if `watched` is false, before any dedup or filter work.
45
46 This is a small migration plus ~5 lines in `telethon_client.py`.
47
48 Discoverability note. With `DEFAULT 0` plus the current lazy population
49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63 used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id   INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id  INTEGER,             -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )

```

```

81     ``
82
83     On a match, look up applicable routing rules by `source_channel_id` and
84     deliver to each matching destination. The delivery layer (`delivery.py`)
85     already accepts a `target` per call, so this generalises the existing single
86     `delivery_target` to a per-match lookup. Users with no routing rules fall
87     back to the current global `delivery_target` behaviour (ADR 0005), preserving
88     backwards compatibility.
89
90     ## Consequences
91
92     **Positive**
93     - Enables the multiplexing product shape: scan curated sources, publish to
94       topical broadcast channels, let subscribers pick outputs by interest.
95     - Each capability is additive and independently deployable; none disturbs the
96       existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97     - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98       file, so a routed re-upload is not re-processed.
99     - Channel enumeration reuses `client.get_dialogs()` already present in
100       `backfill.py`; no new Telethon capability is required.
101
102     **Negative**
103     - New schema (`channels.watched`, `routing_rules`) with migrations.
104     - **Routing loop invariant.** With multiplexed routing, the userbot account
105       may be a member of a *destination* broadcast channel and re-see its own
106       output, producing an infinite forward loop. The contract is: **a channel
107       is either a source or a destination, never both.** This is enforced in
108       `on_new_message` by excluding any channel that is a configured destination
109       (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110        user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111       this; the invariant is stated here so the handler has a contract to code
112       against.
113     - Per-route filter overrides add config complexity; v1 may ship with global
114       filters only and a single implicit route (`source ? global target`), then
115       layer explicit per-route rules later.
116
117     ## Backwards compatibility
118
119     Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120     install from receiving matches until the user re-watches their channels ? a
121     behaviour change disguised as a schema change. To preserve the current
122     "ingest everything" semantics for existing installs, the migration backfills:
123
124     ```sql
125     ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126     UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127     ```
128
129     i.e. any channel the install has *already* seen traffic from is implicitly
130     watched, so pre-migration users observe no change. New channels discovered
131     after the migration default to `watched = 0` and must be explicitly watched ?
132     which is the intended new UX. This mirrors the "old rows behave exactly as
133     before" pattern established by ADR 0005.
134
135     ## Rollout order
136
137     1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138       `/channels`). Smallest unit; immediately useful even without routing.
139     2. **Named destination channels** ? extend `/set delivery_target` to allow
140       switching among a user-defined set, not just one global value.
141     3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142       `/route <source> <dest>` command). The full multiplexing vision.
143
144     Each step is shippable and useful on its own; step 3 is not required to
145     deliver value from steps 1?2.

```

```

146
147 ## Alternatives considered
148
149 - **Server-side `chats=` filter at handler registration:** rejected ? the
150   watched set is per-user and mutable at runtime; re-registering Telethon
151   handlers on every `/watch` change is brittle. An in-handler lookup against
152   the `channels` table is simpler and hot-swappable.
153 - **Content-based classification (auto-detect genre):** explicitly out of
154   scope for v1. Routing is user-defined by source channel; automatic topic
155   detection could be a future enhancement but is not required to deliver the
156   multiplexing UX.
157
158 ## References
159
160 - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161 - ADR 0005 (per-user delivery target) ? the single-target delivery this
162   generalises into per-route targets.
163 - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164   the original `document_id`.
165 - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166   `/unwatch`, and routing commands to public users.
167

```

4. user (2026-07-06T09:13:49.479Z)

```

150     engine_holder: dict[str, FilterEngine],
151     client: TelegramClient,
152     delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153     delivery_target: str = "me",
154 ) -> _NewMessageCallback:
155     """Return an async callback for ``events.NewMessage``.
156
157     The callback:
158     1. extracts VideoMeta + document_id (skips if not a video),
159     2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
160       plus the existing (channel, message) instance dedup,
161     3. runs the filter engine (read from ``engine_holder`` so config
162       changes via ``/set`` take effect immediately),
163     4. inserts a VideoRecord row,
164     5. delivers a match to ``delivery_target`` (see ADR 0005).
165     """
166
167     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
168         message: Message = event.message # type: ignore[assignment]
169         if message is None:
170             return # Telethon edge case: event has no message (shouldn't happen)
171         # Only process genuine channel/group posts and private messages we
172         # receive ? outgoing messages (sent by our own account) are ignored.
173         if getattr(message, "out", False):
174             return
175
176         meta = extract_video_meta(message)
177         if meta is None:
178             return # not a video
179
180         channel_id = _peer_id(message.peer_id)
181         message_id = message.id
182         document_id = _extract_document_id(message)
183
184         # --- watched gate (ADR 0009, Track E) -----
185         # Skip ingestion for unwatched sources. The channel must already be in
186         # sources (lazy-created by the /channels enumeration or the legacy
187         # upsert_channel below); if absent, we let the legacy path upsert it
188         # with watched=True for backwards-compatibility.
189         src = await db.get_source_by_provider_id(conn, user.id, "telegram",

```

```

str(channel_id))
190         if src is not None and not src.watched:
191             return # silently skip unwatched source
192
193         # --- three-tier dedup (cheapest first) ---
194
195         # Tier 0: same (user, channel, message) instance ? already processed.
196         existing = await db.get_video(conn, user.id, channel_id, message_id)
197         if existing is not None:
198             return
199
200         # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
201         if document_id is not None:
202             existing = await db.get_video_by_document_id(conn, user.id, document_id)
203             if existing is not None:
204                 log.info(
205                     "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
206                     "original_channel=%s original_msg=%s",
207                     user.id,
208                     document_id,
209                     channel_id,
210                     message_id,
211                     existing.channel_id,
212                     existing.message_id,
213                 )
214                 return
215
216         # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
217         if (
218             meta.size_bytes is not None
219             and meta.duration_s is not None
220             and meta.height is not None
221             and meta.width is not None
222         ):
223             existing = await db.get_video_by_content_fingerprint(
224                 conn,
225                 user.id,
226                 meta.size_bytes,
227                 meta.duration_s,
228                 meta.height,
229                 meta.width,

```

5. assistant (2026-07-06T09:13:53.055Z)

Both the ADR and the gate are clear. Let me verify the key claims: `upsert\_channel` hardcoding `watched=True`, and the exact control flow. Let me check db.py.

6. user (2026-07-06T09:13:54.078Z)

```

230         )
231         if existing is not None:
232             log.info(
233                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
234                 "channel=%s msg=%s original_channel=%s original_msg=%s",
235                 user.id,
236                 meta.size_bytes,
237                 meta.duration_s,
238                 meta.height,
239                 meta.width,
240                 channel_id,
241                 message_id,
242                 existing.channel_id,
243                 existing.message_id,
244             )
245             return

```

```

246
247     # Lazily record the channel (best-effort; failures are non-fatal).
248     try:
249         await db.upsert_channel(
250             conn,
251             Channel(user_id=user.id, channel_id=channel_id,
252 title=_channel_title(event)),
253         )
254     except Exception:
255         log.warning(
256             "failed to upsert channel %s for user %s", channel_id, user.id,
257 exc_info=True
258         )
259
260     result = engine_holder[ENGINE_KEY].evaluate(meta)
261     record = VideoRecord(
262         user_id=user.id,
263         channel_id=channel_id,
264         message_id=message_id,
265         document_id=document_id,
266         duration_s=meta.duration_s,
267         width=meta.width,
268         height=meta.height,
269         size_bytes=meta.size_bytes,
270         mime_type=meta.mime_type,
271         matched=result.matched,
272     )
273     try:
274         inserted = await db.insert_video(conn, record)
275     except sqlite3.IntegrityError:
276         # Lost a dedup race: a concurrent delivery of the same file
277         # (same document_id or same message instance) won the insert.
278         # The three-tier check above already classified this as new, so
279         # the collision means another task is processing (or has
280         # processed) it ? log and skip.
281         log.info(
282             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
283             user.id,
284             channel_id,
285             message_id,
286             document_id,
287         )
288         return
289     log.info(
290         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
291         user.id,
292         channel_id,
293         message_id,
294         result.matched,
295         result.reasons,
296     )
297
298     if result.matched:
299         # Build link text for LINK/BOTH modes (None for FORWARD).
300         link_text = None
301         if delivery_mode != DeliveryMode.FORWARD:
302             channel_title = _channel_title(event)
303             link_text = build_link_text(
304                 message_link=getattr(message, "link", None),
305                 duration_s=meta.duration_s,
306                 height=meta.height,
307                 size_bytes=meta.size_bytes,
308                 channel_title=channel_title,
309             )
310         await deliver(

```

309 client,

7. user (2026-07-06T09:13:55.366Z)

```
1142:async def get_source_by_provider_id(
1387:async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1537:async def set_source_watched(
```

8. user (2026-07-06T09:13:59.450Z)

```
1387     async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1388         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1389
1390         ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1391         reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1392         New code should call :func:`upsert_source` directly.
1393         """
1394         src = await upsert_source(
1395             conn,
1396             Source(
1397                 account_id=channel.user_id,
1398                 provider="telegram",
1399                 provider_source_id=str(channel.channel_id),
1400                 title=channel.title,
1401                 kind="channel",
1402                 watched=True, # back-compat: pre-ADR-0011 channels were all watched
1403                 seen_first_at=channel.seen_first_at,
1404             ),
1405         )
1406         return _source_to_channel(src)
1407
1408     async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
1410         """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1411         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1412         return _source_to_channel(src) if src else None
1413
1414     def _source_to_channel(src: Source) -> Channel:
1415         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`.
1416
1417         Only valid for Telegram sources (``provider == "telegram"``); the legacy
1418         ``Channel.channel_id`` is a Telegram-specific int id. Non-Telegram sources
1419         raise ``ValueError`` ? new code should use :class:`Source` directly instead
1420         of the deprecated :class:`Channel` shim (ADR 0011 provider-neutrality guard).
1421         """
1422         if src.provider != "telegram":
1423             raise ValueError(
1424                 f"Cannot project {src.provider!r} source onto the Telegram-specific "
1425                 f"Channel type. Use Source directly (ADR 0011)."
1426             )
1427         return Channel(
1428             id=src.id,
1429             user_id=src.account_id,
1430             channel_id=int(src.provider_source_id),
1431             title=src.title,
1432             seen_first_at=src.seen_first_at,
1433         )
1434
1435     # --- legacy VideoRecord shims (backwards-compatible view over content_items) -
1436
1437     async def get_video(
```



```

1441     conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
1442 ) -> VideoRecord | None:
1443     """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1444
1445     Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1446     :class:`Source` + :class:`ContentItem` and projects the latter onto a

```

9. user (2026-07-06T09:14:00.232Z)

```

1142 async def get_source_by_provider_id(
1143     conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144 ) -> Source | None:
1145     """Resolve a source by its provider-native id."""
1146     conn.row_factory = aiosqlite.Row
1147     async with conn.execute(
1148         """SELECT * FROM sources
1149            WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1150         (account_id, provider, provider_source_id),
1151     ) as cur:
1152         row = await cur.fetchone()
1153     return _row_to_source(row) if row else None
1154
1155
1156 # --- ContentItem repos (ADR 0011) -----
1157
1158
1159 async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
1160     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1161
1162     The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1163     unique constraints guard against duplicates:
1164     - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1165     - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1166     """
1167     cur = await conn.execute(
1168         """INSERT INTO content_items
1169            (account_id, source_id, provider, provider_message_id, content_type,
1170             text, link_url, media_ref, document_id, duration_s, width, height,
1171             size_bytes, mime_type, dedup_key, matched, seen_at)
1172            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
1173         (
1174             item.account_id,
1175             item.source_id,
1176             item.provider,
1177             item.provider_message_id,
1178             item.content_type,
1179             item.text,
1180             item.link_url,
1181             item.media_ref,

```

10. assistant (2026-07-06T09:14:08.839Z)

The finding's factual claims are confirmed. The gate at telethon_client.py:190 is `if src is not None and not src.watched: return`. When `src is None` (brand-new channel never seen), the gate falls through, the message is deduped/filtered/delivered, and then `upsert\_channel` at line 249 creates the source with `watched=True` (db.py:1402).

Now the critical adversarial question: is `str(channel\_id)` the same key on both the lookup (line 189) and the upsert (line 1399)? Both use `str(channel\_id)` / `str(channel.channel\_id)` with provider "telegram" ? yes, same key. So a truly new channel: lookup returns None, processed+delivered, then upserted as watched=True. Every subsequent message from that channel: lookup now returns watched=True, gate stays False, keeps scanning.

Let me verify against ADR whether this deviates. ADR line 130-131 is explicit: "New channels

discovered after the migration default to watched = 0 and must be explicitly watched." The code does the opposite. Let me run a repro to be certain the control flow behaves as claimed, since this is the load-bearing claim.

11. user (2026-07-06T09:14:10.387Z)

```
1537 async def set_source_watched(
1538     conn: aiosqlite.Connection, account_id: int, source_id: int, watched: bool
1539 ) -> None:
1540     """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
1541     await conn.execute(
1542         "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
1543         (int(watched), source_id, account_id),
1544     )
1545     await conn.commit()
1546
1547
1548 async def list_account_sources(
1549     conn: aiosqlite.Connection, account_id: int, *, watched_only: bool = False
1550 ) -> list[Source]:
1551     """List all sources for an account, optionally filtering to watched only."""
1552     conn.row_factory = aiosqlite.Row
1553     query = "SELECT * FROM sources WHERE account_id = ?"
1554     params: tuple = (account_id,)
1555     if watched_only:
1556         query += " AND watched = 1"
1557     query += " ORDER BY watched DESC, provider_source_id ASC"
1558     async with conn.execute(query, params) as cur:
1559         rows = await cur.fetchall()
1560     return [_row_to_source(r) for r in rows]
1561
1562
1563 async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
1564     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1565
1566     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1567     if absent), computes the provider-neutral ``dedup_key`` via
1568     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1569     ``VideoRecord`` shape is reconstructed from the inserted row.
1570     """
1571     src = await upsert_source(
1572         conn,
1573         Source(
1574             account_id=video.user_id,
1575             provider="telegram",
1576             provider_source_id=str(video.channel_id),
```

12. user (2026-07-06T09:14:11.779Z)

```
99:     watched            INTEGER NOT NULL DEFAULT 0,
522:     # 1a. Backfill sources from channels first (known title + watched=1). Order
523:     #     matters: channels-first so title/watched data wins; the videos-based
528:         (account_id, provider, provider_source_id, title, kind, watched,
539:     #     ? the channels-based row above survives with its title + watched=1.
543:         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
851:     watched=bool(row["watched"]),
1105:     """Insert a source row, or update title/watched on conflict.
1112:         (account_id, provider, provider_source_id, title, kind, watched,
1117:         watched=excluded.watched,
1125:         int(source.watched),
1402:         watched=True, # back-compat: pre-ADR-0011 channels were all watched
1537:async def set_source_watched(
1538:     conn: aiosqlite.Connection, account_id: int, source_id: int, watched: bool
1540:     """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
1542:     "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
```

```
1543:         (int(watched), source_id, account_id),
1549:     conn: aiosqlite.Connection, account_id: int, *, watched_only: bool = False
1551:     """List all sources for an account, optionally filtering to watched only."""
1555:     if watched_only:
1556:         query += " AND watched = 1"
1557:     query += " ORDER BY watched DESC, provider_source_id ASC"
1578:         watched=True,
1746:     "set_source_watched",
```

13. assistant (2026-07-06T09:14:29.747Z)

Important additional evidence: ``upsert_source`` (line 1105-1117) does ``ON CONFLICT ... watched=excluded.watched``. So even if a discoverability row existed as `watched=0`, the lazy ``upsert_channel`` at `telethon_client.py:249` with ``watched=True`` would **OVERWRITE** it back to `watched=1` on the very next message. And ``insert_video`` (line 1571-1578) also upserts the source with ``watched=True``. This compounds the finding.

But wait ? for a channel that already has a `watched=0` row, the gate at line 190 returns early **BEFORE** reaching the `upsert_channel` at line 249, so that overwrite only bites brand-new (`src is None`) channels. For those, the finding holds fully.

Now let me confirm the control-flow claim is real and not blocked by the ``# pragma: no cover`` wrapper ? that pragma just excludes it from coverage, the code still runs live. The finding is about the live ingestion path.

Let me trace whether there's any earlier destination-exclusion or watched enforcement I might be missing above line 189. I've read lines 167-191; the gate is the only watched check, and there's no ``src is None -> skip`` branch. Confirmed.

Let me verify the ``_apply_schema`` default is 0 (line 99) to confirm a fresh discoverability row would be opt-in ? yes, ``watched INTEGER NOT NULL DEFAULT 0``. So the schema is correctly opt-in, but the ingestion path's lazy upsert forces `watched=True`, defeating it.

The finding is correct on all counts. Let me assess severity. The ADR frames "scan nothing until opt-in" as the locked default (line 37-39) and "new channels default to `watched=0`" as the intended new UX (line 130-131). The gate inverts this: any new channel is auto-scanned, auto-delivered on first message, and auto-watched thereafter. This is a direct violation of the ADR's central selective-scanning contract ? the very thing Track E exists to implement. The feature is effectively non-functional for its stated purpose (a freshly-onboarded user receives matches from everything).

One nuance on severity: this isn't "data loss on the live path" in the destructive sense. It's a feature-contract inversion ? the opt-in gate does not actually gate new sources. Users hit it immediately (any new channel). That is a real bug users hit and a direct ADR-contract violation ? high, arguably blocker. The finding's "blocker" rests on "feature broken." The core selective-scanning promise (scan nothing until opt-in) is broken for all new channels, which is the majority real-world case for a fresh user. I'll rate it blocker: the feature's primary guarantee is inverted on the live ingestion path.

14. user (2026-07-06T09:14:39.757Z)

Structured output provided successfully